



06 — Retrieval Pipeline

Overview

The retrieval pipeline combines four complementary mechanisms:

1. **Cosine-direct retrieval** — exact FAISS search over episodic embeddings (literal recall).
2. **Spreading activation** — iterative propagation over the prototype graph (pattern completion).
3. **Predictive completion** — transition model foresees the next-state embedding as a second seed (sequential reasoning).
4. **Multi-scale retrieval** — parallel fine (CA3) and coarse (CA1) prototype seeds with co-occurrence bonus (dual-level evidence).

Results are merged with a score ceiling, temporally boosted, salience-re-ranked, and diversity-capped.

Mathematical Formulation

Phase 1: Query Encoding

$$\mathbf{q} = \text{encode}(\text{query}) \in \mathbb{R}^d, \quad \|\mathbf{q}\|_2 = 1$$

The pipeline receives an already-encoded L2-normalized query vector.

Phase 2: Cosine-Direct Retrieval

FAISS search over episodic embeddings:

$$\mathcal{C} = \{(i, s_i) \mid i \in \text{FAISS.search}(\mathbf{q}, k_e), s_i = \langle \mathbf{q}, \mathbf{e}_i \rangle\}$$

Where $k_e = \text{episodic_top_k}$ (default: `10`).

Fine-scale prototype search (CA3-like, narrow precise matches):

$$\mathcal{P}_{\text{seed}} = \{(j, s_j) \mid j \in \text{SemanticStore.search}(\mathbf{q}, k_s), s_j = \langle \mathbf{q}, \mathbf{c}_j \rangle\}$$

Where $k_s = \text{semantic_top_k}$ (default: 6).

Phase 2a: Multi-Scale Prototype Seeds (Stage 9)

When `use_multi_scale = True`, a second coarse-scale (CA1-like) search runs in parallel on prototypes assigned with a lower similarity threshold. Coarse prototypes capture broader topics; fine prototypes capture narrow patterns:

$$\mathcal{P}_{\text{coarse}} = \{(j, s_j) \mid j \in \text{SemanticStore.search_by_scale}(\mathbf{q}, \text{coarse}, k_c)\}$$

Coarse seeds are max-merged into the seed activation set, and their member episodes are later checked for dual-scale co-occurrence (see Phase 6).

Phase 2b: Predictive Completion (Stage 3)

When `use_transition = True` and the transition model is trained, a forward prediction $\hat{\mathbf{q}} = \text{TransitionModel.predict}(\mathbf{q})$ is computed. If $\|\hat{\mathbf{q}}\|_2 \geq \text{transition_min_norm}$ (default 10^{-2}), it acts as a second cosine seed:

$$s_i^{\text{pred}} = w_{\text{trans}} \cdot \langle \hat{\mathbf{q}}, \mathbf{e}_i \rangle, \quad w_{\text{trans}} = \text{transition_score_weight} (0.7)$$

Predicted scores are max-merged with cosine-direct scores: a strong literal match is never demoted by a weaker prediction.

A **reserved-slot mechanism** reserves up to `transition_reserved_slots` (default 1) head positions for predictive-seed candidates that would otherwise be displaced by cosine-direct episodes — but only when the prediction direction is *meaningfully different* from the query: $\langle \mathbf{q}, \hat{\mathbf{q}} \rangle \leq \text{transition_reserve_max_qsim}$ (default 0.85).

Phase 3: Spreading Activation (Stage 1)

When `use_spreading = True`, activation propagates from seed prototypes over the graph.

Initialization: seed prototypes receive activation proportional to their cosine match:

$$a_0(p) = \frac{\max(0, \langle \mathbf{q}, \mathbf{c}_p \rangle)}{\max_j \langle \mathbf{q}, \mathbf{c}_j \rangle}$$

Propagation (for `spread_steps` iterations):

$$a_{t+1}(p) = \alpha \cdot a_t(p) + (1 - \alpha) \sum_{q \in \mathcal{N}_{\text{in}}(p)} \frac{w_{qp}}{\sum_r w_{qr}} \cdot a_t(q)$$

Where:

- $\alpha = \text{spread_decay}$ (default: `0.6`) — retention factor (60% self-retention, 40% from neighbors)
- $\mathcal{N}_{\text{in}}(p)$ = prototypes with outgoing edges to p
- w_{qp} = stored edge weight (fused similarity + transition + coactivation)

This update rule is structurally identical to **personalized PageRank** (random walk with restart). α controls the trade-off: α near 0 means activation spreads far but fast (everything gets weakly activated — noise), α near 1 means activation barely moves (effectively cosine-only).

Pruning: below `spread_activation_floor` (default: `10^{-3}`), activation is zeroed each step. This enforces **sparse coding** — without it, every prototype reachable in $n+1$ hops would acquire activation, growing the front unboundedly. Normalization per source (L1 over outgoing weights) prevents super-hub domination.

Saliency gate (optional, `saliency_gate=True`): final activation modulated by prototype support:

$$a_{\text{final}}(p) = a(p) \cdot (1 + 0.1 \cdot \sqrt{1 + \text{support_count}(p)})$$

This is a **Hebbian prior**: prototypes with more evidence (higher support count from replay/consolidation) are modestly easier to activate. The $\sqrt{}$ dampens returns — a prototype with 10,000 episodes is only $\sim 3.2\times$ more activatable than one with 1 episode. The constant 0.1 keeps this a soft prior, never a hard filter.

Phase 4: Episode Harvesting and Predictive Promotion

From each activated prototype, harvest member episodes:

$$s_{\text{harvest}}(e) = \min(1, a(p)) \cdot w_{\text{spread}} \cdot (0.5 + 0.5 \cdot \text{saliency}(e))$$

Where $w_{\text{spread}} = \text{spread_episode_weight}$ (default: `0.15`). The saliency multiplier (range

[0.5, 1.0]) gives higher-salience episodes within the harvest a modest boost — frequently recalled patterns surface more easily.

Score ceiling: graph-harvested scores are hard-capped. This preserves cosine-direct as the **trust anchor** — graph edges can be noisy, and a spurious edge must never promote an irrelevant episode:

$$s_{\text{harvest}}(e) \leq \gamma_{\text{ceiling}} \cdot \min_{e' \in \mathcal{C}} s_{e'}$$

Where $\gamma_{\text{ceiling}} = \text{spread_score_ceiling}$ (default: 0.9).

Diversity cap: at most `diversity_per_prototype` (default: 2) episodes per prototype from graph harvest. Cosine-direct episodes are **exempt** — they always retain full representation. The cap prevents a single highly-activated prototype from flooding the head.

Predictive-seed promotion: episodes shared by both cosine-direct and predictive seeds are promoted to the head when the predictive head is otherwise empty, up to `transition_reserved_slots` positions.

Phase 5: Temporal Boost (Stage 7)

For each candidate episode e with timestamp t_e :

$$s_{\text{temporal}}(e) = \alpha_t \cdot \cos(\mathbf{t}_q, \mathbf{t}_e)$$

Where:

- $\mathbf{t}_q$ = temporal embedding of the query anchor timestamp
- $\mathbf{t}_e$ = temporal embedding of the episode timestamp (multi-scale sinusoidal, see §07)
- α_t = `temporal_weight` (default: 0.25)

Temporal boost is **additive, not multiplicative** — it nudges ranking by at most 25% of a perfect cosine match, never dominating semantic relevance.

Phase 6: Salience Re-Ranking

Final score for each episode combines three independent signals additively:

$$s_{\text{final}}(e) = \underbrace{\cos(\mathbf{q}, \mathbf{e})}_{\text{semantic}} + \underbrace{\alpha_t \cdot \cos(\mathbf{t}_q, \mathbf{t}_e)}_{\text{temporal}} + \underbrace{\alpha_s \cdot s_{\text{salience}}(e)}_{\text{salience}}$$

Where $\alpha_s = \text{salience_weight}$ (default: `0.3`). For graph-harvested episodes, the semantic component is replaced by the harvest score (capped).

Multi-scale co-occurrence bonus: when an episode appears in both fine-scale and coarse-scale results, its final score is multiplied by $1 + \omega$ where $\omega = \text{multi_scale_co_occurrence_bonus}$ (default: `0.25`). Dual-scale evidence is stronger: the episode is confirmed by two independent retrieval pathways.

Temporal anchor override: the query temporal anchor defaults to "now" (recency bias), but can be explicitly set via `temporal_anchor_ts` for past-anchored queries (e.g., "what did we know last month").

Phase 7: Reinforcement

Only cosine-direct episodes in the top slice receive recall reinforcement (+ `salience_weight` to salience, +1 to `recalled_count`). Graph-harvested episodes do **not** receive reinforcement — this prevents self-rewarding loops where graph-activated prototypes accumulate salience independently of relevance.

Configuration

RetrievalConfig

Parameter	Type	Default	Description
<code>episodic_top_k</code>	<code>int</code>	<code>10</code>	Cosine-direct episodes to fetch
<code>semantic_top_k</code>	<code>int</code>	<code>6</code>	Cosine-direct prototypes to fetch
<code>neighbor_top_k</code>	<code>int</code>	<code>6</code>	Neighbors per prototype

Parameter	Type	Default	Description
			in propagation
<code>salience_weight</code> α_s	float	0.3	Saliency contribution to final score
<code>use_spreading</code>	bool	True	Enable spreading activation
<code>spread_steps</code>	int	2	Propagation iterations
<code>spread_decay</code> α	float	0.6	Activation retention per step
<code>spread_activation_floor</code>	float	10^{-3}	Pruning threshold
<code>episodes_per_prototype</code>	int	6	Episodes harvested per prototype
<code>spread_episode_weight</code>	float	0.15	Base score for graph-harvested episodes
<code>spread_score_ceiling</code>	float	0.9	Max harvest score relative to cosine
<code>salience_gate</code>	bool	True	Enable support-count modulation
<code>diversity_per_prototype</code>	int	2	Max graph episodes per prototype
<code>use_temporal</code>	bool	True	Enable temporal proximity bonus
<code>temporal_weight</code> α_t	float	0.25	Temporal contribution to final score
<code>temporal_anchor_ts</code>	int None	None	Explicit query anchor (Unix ts); <code>None</code> = "now"
<code>use_multi_scale</code>	bool	True	Enable dual-scale (fine +

Parameter	Type	Default	Description
			coarse) search
<code>coarse_semantic_top_k</code>	int	6	Coarse-scale (CA1) prototype seeds
<code>multi_scale_co_occurrence_bonus</code> ω	float	0.25	Multiplicative bonus for dual-scale episodes
<code>use_transition</code>	bool	True	Enable transition model predictive completion
<code>transition_top_k</code>	int	6	Predicted-seed episodes to fetch
<code>transition_score_weight</code> w_{trans}	float	0.7	Discount for predicted-seed scores
<code>transition_min_norm</code>	float	10^{-2}	Min predicted-embedding norm to activate
<code>transition_reserved_slots</code>	int	1	Head slots reserved for predictive candidates
<code>transition_reserve_max_qsim</code>	float	0.85	Max q-pred similarity to trigger reserved slot

Key Invariants

1. Cosine-direct episodes always outrank graph-harvested ones (score ceiling + low base weight).
2. Spreading activation solves pattern completion: episodes with weak cosine matches can still be retrieved if reachable through high-weight edges.
3. Only cosine-direct episodes receive reinforcement — prevents graph self-reinforcement.
4. Temporal boost is additive, not multiplicative — it nudges ranking rather than dominating.

5. Saliency gate modestly boosts well-consolidated prototypes (Hebbian: "fire together, wire together").
6. Multi-scale co-occurrence bonus ($\times 1.25$) rewards episodes independently confirmed by both CA3 and CA1 pathways.
7. Predictive completion respects a hard similarity gate: predictions too similar to the query ($\cos > 0.85$) don't trigger reserved slots — it's not pattern completion if you're predicting what you already see.
8. Graph-harvested episodes inherit the score ceiling from the worst cosine-direct episode, ensuring "cosine-direct > graph" hierarchy regardless of edge quality.
9. Diversity cap exempts cosine-direct episodes — they are never displaced by diversity enforcement.